

lux Connection-Lease Reaping — Fidelity Control (No Release on Departure)

Buggy variant: Depart never releases the departing connection's scenes

September 2026

1 Introduction

This is the fidelity control for H2 in `docs/connection_lease_reaping.tex`, not the design. It removes only the release half of **Depart**: a departing connection – graceful disconnect or SDK idle-reap, no lease condition required – is still correctly dropped from **registered**, but every scene it owned is left exactly as it was. This models `HubDisplay.drop_connection`'s own docstring verbatim: “a session's UI survives the session: the connection's roots stay installed and stay owned by its id.” Read the main specification first; this document states only what differs, in Section 6.1.

2 Basic Types

$[CONN, IDENT, SCENE]$

Unchanged from the main specification.

3 Free Types

$TSTATE ::= tup \mid tdown$

$LSTATE ::= llive \mid llapsed$

$OWNER ::= unowned \mid conn\langle\langle CONN \rangle\rangle$

Unchanged from the main specification.

4 State

ConnReg

$registered : \mathbb{P} CONN$
 $identity : CONN \leftrightarrow IDENT$
 $transport : CONN \leftrightarrow TSTATE$
 $lease : CONN \leftrightarrow LSTATE$
 $owner : SCENE \rightarrow OWNER$

$registered \subseteq \text{dom } identity$
 $\text{dom } identity = \text{dom } transport$
 $\text{dom } transport = \text{dom } lease$

Unchanged from the main specification.

5 Initialization

<i>Init</i>
<i>ConnReg'</i>
$registered' = \emptyset$
$identity' = \emptyset$
$transport' = \emptyset$
$lease' = \emptyset$
$owner' = (\lambda s : SCENE \bullet unowned)$

Unchanged from the main specification.

6 Operations

6.1 Depart — BUGGY: no ownership release

This is the one operation that differs from the design. The departing connection $c?$ still correctly leaves **registered** – no lease condition guards it, matching the real **drop_connection**, which fires on an ordinary clean disconnect regardless of lease state – but **owner** is left completely unchanged: no scene $c?$ owned is released. Compare against the correct **Depart** in `connection_lease_reaping.tex`: the correct version clears every scene owned by $c?$ to **unowned** in the same step this one only drops $c?$ from **registered**.

<i>Depart</i>
$\Delta ConnReg$
$c? : CONN$
$c? \in registered$
$registered' = registered \setminus \{c?\}$
$owner' = owner$
$identity' = identity$
$transport' = transport$
$lease' = lease$

6.2 Connect, Renew, TransportDies, LeaseLapse, Read, TimedReap, Claim — unchanged

Identical to the main specification: only **Depart**'s ownership-release step is removed. **TimedReap** in particular is the *correct*, releasing coordinator – this control isolates the H2 finding to **Depart**'s own graceful-disconnect path, so the background backstop must stay intact here exactly as it is in the fixed spec.

<i>Connect</i>
$\Delta ConnReg$
$c? : CONN$
$i? : IDENT$
$registered' = registered \cup \{c?\}$
$identity' = identity \oplus \{c? \mapsto i?\}$
$transport' = transport \oplus \{c? \mapsto tup\}$
$lease' = lease \oplus \{c? \mapsto llive\}$
$owner' = owner$

Renew

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $lease' = lease \oplus \{c? \mapsto llive\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

TransportDies

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $transport' = transport \oplus \{c? \mapsto tdown\}$
 $registered' = registered$
 $identity' = identity$
 $lease' = lease$
 $owner' = owner$

LeaseLapse

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llive$
 $lease' = lease \oplus \{c? \mapsto llapsed\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

Read

$\Xi ConnReg$

TimedReap

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llapsed$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $\quad (\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$

Claim

$\Delta ConnReg$

$c? : CONN$

$s? : SCENE$

$c? \in registered$

$transport\ c? = tup$

$lease' = lease \oplus \{c? \mapsto llive\}$

$registered' = registered \setminus$

$\{c2 : CONN \mid c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed\}$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\}))\ s? = unowned$

$\vee$

$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\}))\ s? = conn(c?)$

$owner' = (owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$

$c2 \in registered \wedge c2 \neq c? \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$

$\times \{unowned\})) \oplus \{s? \mapsto conn(c?)\}$

$identity' = identity$

$transport' = transport$

7 Verification: the goal that must flip

Against this variant, I2/I5's negation must be *found* — the trace in `connection_lease_reaping.tex`, Section H2, reaches it directly, and never lapses a lease at all: `Connect(c1, i1); Claim(c1, s1); Depart(c1)`.

```
# must be FOUND against this variant (NOT found against the fixed spec)
probc1 docs/connection_lease_reaping_no_release_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered)))" \
-p DEFAULT_SETSIZE 3
```

I3's negation must also be *found*, continuing the same trace one step further with a live reconnect under the same identity – `Connect(c2, i1)`:

```
# must be FOUND against this variant (NOT found against the fixed spec)
probc1 docs/connection_lease_reaping_no_release_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(#c2.(s:SCENE & c:CONN & c2:CONN &
      c : registered & not(c2 : registered) &
      c2 : dom(identity) & identity(c) = identity(c2) &
      not(c = c2) & owner(s) = conn(c2))))" \
-p DEFAULT_SETSIZE 3
```

I1 must stay unaffected — this variant does not touch `TransportDies` or `LeaseLapse`, so the chain `TransportDies; Depart` still completes exactly as in the fixed spec (this variant does not remove `Depart`'s own guard, only what happens to `owner` afterward):

```
# must remain FOUND
probc1 docs/connection_lease_reaping_no_release_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
      transport(c) = tdown)" \
-p DEFAULT_SETSIZE 3
```